

Compiler Design Interview Guide

This guide is written for SDE placements, technical interviews, viva preparation, and online assessment revision. Each topic moves from basic meaning to interview-level depth, with examples, diagrams, comparisons, mistakes, edge cases, and practice tasks.

Compiler vs Interpreter

1. Overview

Definition

A **compiler** translates an entire source program into target code, usually machine code, bytecode, or intermediate code, before execution.

An **interpreter** executes source code or intermediate code step by step, usually translating and running one statement or instruction at a time.

Why It Matters

This topic matters because every programming language implementation must decide how code becomes executable. The choice affects:

Factor	Impact
Speed	Compiled code usually runs faster after translation
Startup time	Interpreters can start quickly
Error detection	Compilers can catch many errors before execution
Portability	Interpreters and bytecode VMs improve portability
Debugging	Interpreters often provide easier interactive debugging

Where It Is Used in Real Systems

System	Example
Operating systems	C/C++ compiled to native binaries
Browsers	JavaScript interpreted and JIT-compiled
Backend servers	Java, Python, Go, Node.js
Mobile apps	Kotlin/Java bytecode, Swift compiled code
Databases	SQL query compilation and interpretation

Why Interviewers Ask

Interviewers ask this to check whether you understand the basic execution model of programming languages. It also tests if you know trade-offs, not just definitions.

2. Core Idea

Intuition

A compiler is like translating an entire book before reading it. An interpreter is like translating each sentence while reading.

Real-World Analogy

Tool	Analogy
Compiler	Translator converts entire English book to Hindi first
Interpreter	Live translator translates each sentence during a meeting

Small Example

Source code:

```
int x = 10;
int y = x + 5;
printf("%d", y);
```

A compiler checks the complete program, produces executable code, and then the CPU runs it.

An interpreter reads each line or instruction, understands it, and executes it immediately.

Step-by-Step Explanation

Compiler flow:

```
Source Code -> Compiler -> Machine Code/Bytecode -> Execution
```

Interpreter flow:

```
Source Code -> Interpreter -> Execute statement by statement
```

Modern systems often combine both:

```
Java Source -> javac Compiler -> Bytecode -> JVM Interpreter/JIT -> Machine Code
```

3. Important Subtopics

Ahead-of-Time Compilation

AOT compilation translates code before the program runs.

Why it matters: it improves runtime performance and catches errors early.

Example: C, C++, Go, Rust.

Common interview angle: "Why is C usually faster than Python?"

Interpretation

Interpretation executes code during runtime.

Why it matters: it supports dynamic behavior, REPLs, and portability.

Example: Python source is compiled to bytecode and then executed by the Python virtual machine.

Common interview angle: "Is Python purely interpreted?"

Bytecode

Bytecode is intermediate code executed by a virtual machine.

Why it matters: it balances portability and performance.

Example: Java `.class` files contain bytecode.

Common interview angle: "Why is Java called platform-independent?"

JIT Compilation

Just-In-Time compilation converts code to machine code during runtime.

Why it matters: it optimizes hot code paths using runtime information.

Example: JVM, V8 JavaScript engine, .NET CLR.

Common interview angle: "How can JavaScript become fast in browsers?"

4. Real-World Example

In a browser, JavaScript is not simply interpreted line by line forever. Modern engines like V8 parse JavaScript, generate intermediate representations, interpret initially, detect frequently executed code, and JIT-compile hot paths into optimized machine code.

This gives fast startup and high performance.

5. Diagrams / Mental Models

Compiler:

```
program.c
  |
  v
Compiler
  |
  v
program.exe
  |
  v
Runs directly on CPU
```

Interpreter:

```
program.py
  |
  v
Interpreter
  |
  v
Executes code while reading it
```

6. Common Interview Questions

1. What is the difference between compiler and interpreter?

Answer: A compiler translates the whole program before execution, while an interpreter executes code step by step during runtime.

Expected points: translation time, runtime speed, error detection, examples.

Common mistake: saying interpreted languages are always slow.

2. Is Java compiled or interpreted?

Answer: Java is both. Source code is compiled into bytecode by `javac`, and bytecode is interpreted or JIT-compiled by the JVM.

Expected points: bytecode, JVM, portability.

Common mistake: calling Java purely compiled or purely interpreted.

3. Why is compiled code usually faster?

Answer: It is already translated into low-level code and can be optimized before execution.

Expected points: optimization, native code, no repeated parsing.

Common mistake: ignoring JIT compilers.

4. Why are interpreters useful?

Answer: They support quick execution, dynamic features, interactive shells, and portability.

Expected points: REPL, debugging, dynamic typing.

Common mistake: saying interpreters are outdated.

5. What is bytecode?

Answer: Bytecode is an intermediate instruction format executed by a virtual machine.

Expected points: JVM, Python VM, portability.

Common mistake: treating bytecode as source code.

6. What is JIT compilation?

Answer: JIT compilation converts code into machine code during execution, usually for frequently executed code.

Expected points: runtime optimization, hot paths.

Common mistake: thinking JIT means no interpretation happens.

7. Which catches errors earlier?

Answer: A compiler usually catches syntax and many semantic errors before execution. Interpreters may detect errors only when that code path runs.

Expected points: compile-time vs runtime errors.

Common mistake: saying interpreters do not catch syntax errors.

8. Why is Python slower than C?

Answer: Python has dynamic typing, runtime checks, object overhead, and bytecode execution. C is compiled to native machine code with static types.

Expected points: dynamic typing, native code, runtime overhead.

Common mistake: blaming only interpretation.

9. Can a language have both compiler and interpreter?

Answer: Yes. Java, C#, JavaScript, and Python implementations use combinations of compilation, bytecode, interpretation, and JIT.

Expected points: implementation vs language distinction.

Common mistake: treating language and implementation as the same thing.

10. What is the main trade-off?

Answer: Compilers favor optimized execution and early checks; interpreters favor flexibility, portability, and fast feedback.

Expected points: balanced comparison.

Common mistake: saying one is always better.

7. Deep-Dive Questions

1. Is "compiled language" a fully accurate term?

Not always. Compilation or interpretation is usually a property of language implementation, not the language itself. For example, JavaScript can be interpreted or JIT-compiled.

2. Why do virtual machines exist?

Virtual machines provide portability, memory management, runtime checks, and optimization opportunities.

3. How does JIT use runtime information?

It observes actual types, branches, and hot functions, then generates optimized machine code for common cases.

4. Can interpreted programs be optimized?

Yes. Interpreters can use bytecode, inline caching, profiling, and JIT compilation.

5. Why do compilers produce intermediate code?

Intermediate code makes optimization easier and allows multiple source languages or target machines to share compiler infrastructure.

8. Comparison Tables

Feature	Compiler	Interpreter
Translation	Whole program/module before execution	During execution
Output	Executable, object code, bytecode, IR	Usually no separate executable
Runtime speed	Usually faster	Usually slower unless JIT optimized
Error detection	Earlier	Often when code executes
Portability	Target-dependent unless bytecode	High if interpreter exists
Examples	C, C++, Go, Rust	Python, Ruby, shell

9. Common Mistakes

- Saying compiler converts only to machine code. It may also generate bytecode or IR.

- Saying interpreter never compiles. Many interpreters compile source to bytecode internally.
- Saying Java is only interpreted.
- Saying Python has no compilation stage.
- Saying compiled always means faster in every case.

10. Edge Cases / Special Cases

- Java uses both compilation and interpretation/JIT.
- JavaScript engines use parsing, bytecode, interpretation, and JIT.
- Python compiles source to bytecode before VM execution.
- Some compilers run at runtime, such as JIT compilers.
- Some interpreters cache bytecode.

11. How to Explain in Interview

"A compiler translates code before execution, usually producing machine code or bytecode, while an interpreter executes code step by step at runtime. Compilers usually give better runtime performance and early error detection, while interpreters give flexibility and fast feedback. Modern languages often combine both, like Java with bytecode and JVM JIT."

12. Quick Revision Notes

- Compiler: translates before execution.
- Interpreter: executes during runtime.
- Bytecode: VM-friendly intermediate code.
- JIT: runtime compilation of hot code.
- Trap: Java and Python are not purely one category.

13. Practice Tasks

1. Compile a small C program and run the executable.
2. Run a Python script and inspect generated `__pycache__`.
3. Explain Java execution from `.java` to JVM.
4. Compare startup time and runtime speed for C and Python examples.
5. Draw the flow of JavaScript execution inside a browser.

14. Final Cheat Sheet

Item	Answer
Core definition	Compiler translates before execution; interpreter executes during runtime
Why it matters	Affects speed, portability, debugging, and error detection
Most asked questions	Java compiled/interpreted, bytecode, JIT, Python execution
Common comparison	Compiler vs Interpreter
One-line answer	Compiler prepares code first; interpreter runs it as it reads it

Phases of a Compiler

1. Overview

Definition

The **phases of a compiler** are the major steps used to translate source code into target code.

Typical phases:

```
Lexical Analysis -> Syntax Analysis -> Semantic Analysis -> Intermediate Code  
Generation -> Optimization -> Code Generation
```

Why It Matters

Compiler phases divide a complex problem into manageable stages. Each phase handles one kind of information:

Phase	Handles
Lexer	Words/tokens
Parser	Grammar structure
Semantic analyzer	Meaning
IR generator	Machine-independent representation
Optimizer	Improved code
Code generator	Target machine code

Where It Is Used in Real Systems

Compiler phases appear in C/C++, Java, TypeScript, Rust, Go, Swift, SQL engines, shader compilers, and browser JavaScript engines.

Why Interviewers Ask

This is a foundation question. It checks whether you understand how source code is processed from text to executable behavior.

2. Core Idea

Intuition

A compiler works like a factory assembly line. Raw source code enters one end, and executable code comes out the other end.

Real-World Analogy

Building a house:

Compiler Phase	House Analogy
Lexical analysis	Identify bricks, cement, wood
Parsing	Check blueprint structure
Semantic analysis	Check whether design is valid
Optimization	Reduce waste
Code generation	Build final house

Small Example

Input:

```
x = a + b * 2;
```

Possible flow:

```
Characters: x = a + b * 2 ;
Tokens: ID ASSIGN ID PLUS ID MUL NUM SEMI
Parse tree: assignment expression
Semantic check: are x, a, b declared?
IR: t1 = b * 2; t2 = a + t1; x = t2
Optimized IR: maybe same
Target code: MOV, MUL, ADD instructions
```

Step-by-Step Explanation

- 1. Read characters.
- 2. Group characters into tokens.
- 3. Check grammatical structure.
- 4. Check meaning and types.
- 5. Generate intermediate representation.
- 6. Optimize representation.
- 7. Generate target code.

3. Important Subtopics

Lexical Analysis

Converts characters into tokens.

Why it matters: removes whitespace/comments and simplifies parsing.

Example: `int x = 5;` becomes `KEYWORD ID ASSIGN NUM SEMI`.

Interview angle: token vs lexeme.

Syntax Analysis

Checks whether tokens follow grammar rules.

Why it matters: detects syntax errors.

Example: `x = + ;` fails grammar.

Interview angle: parse tree, LL/LR parsers.

Semantic Analysis

Checks meaning beyond grammar.

Why it matters: grammar may allow meaningless code.

Example: `int x; x = "hello";` may be syntactically valid but semantically wrong.

Interview angle: type checking, scope checking.

Intermediate Code Generation

Creates machine-independent code.

Why it matters: easier optimization and retargeting.

Example: three-address code.

Interview angle: why use IR?

Code Optimization

Improves performance or reduces code size.

Why it matters: generated code should be efficient.

Example: replacing `x = 2 * 3` with `x = 6`.

Interview angle: dead-code elimination, constant folding.

Code Generation

Produces target machine or bytecode instructions.

Why it matters: final executable behavior.

Example: x86, ARM, JVM bytecode.

Interview angle: register allocation.

Symbol Table

A data structure storing identifiers and their properties.

Why it matters: used by many phases.

Example: variable name, type, scope, memory offset.

Interview angle: why symbol table is central.

4. Real-World Example

In a database, SQL queries go through similar phases:

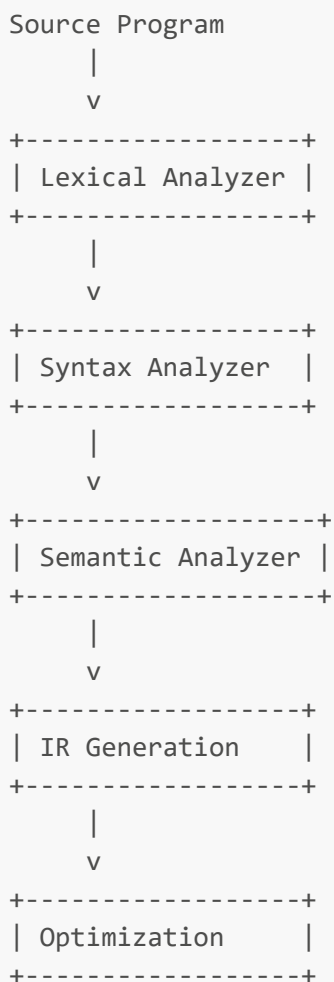
```
SQL text -> Tokens -> Parse tree -> Semantic checks -> Logical plan -> Optimized
plan -> Execution plan
```

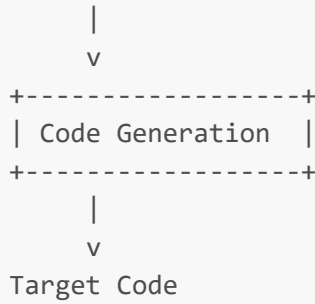
For example:

```
SELECT name FROM users WHERE age > 18;
```

The database parses the query, checks if `users`, `name`, and `age` exist, optimizes access paths using indexes, and executes a plan.

5. Diagrams / Mental Models





6. Common Interview Questions

1. What are the phases of a compiler?

Answer: Lexical analysis, syntax analysis, semantic analysis, intermediate code generation, optimization, and code generation.

Expected points: mention symbol table and error handling.

Common mistake: skipping semantic analysis.

2. What does lexical analysis do?

Answer: It converts character streams into tokens.

Expected points: tokens, lexemes, whitespace removal.

Common mistake: saying it checks grammar.

3. What does syntax analysis do?

Answer: It checks token sequence against grammar and builds parse tree/AST.

Expected points: grammar, parser, parse tree.

Common mistake: mixing parsing with type checking.

4. What is semantic analysis?

Answer: It checks meaning, such as types, declarations, scope, and valid operations.

Expected points: type checking and symbol table.

Common mistake: saying syntax and semantics are the same.

5. Why is intermediate code generated?

Answer: It separates frontend and backend and makes optimization easier.

Expected points: portability, optimization, retargeting.

Common mistake: saying it is always machine code.

6. What is code optimization?

Answer: Transforming code to improve speed, memory use, or size without changing meaning.

Expected points: semantic preservation.

Common mistake: assuming optimization changes output.

7. What is code generation?

Answer: It converts IR into target instructions.

Expected points: instruction selection, register allocation.

Common mistake: ignoring target architecture.

8. What is the role of the symbol table?

Answer: It stores information about identifiers, such as type, scope, and memory location.

Expected points: used across phases.

Common mistake: saying only parser uses it.

9. What is compiler frontend?

Answer: The part that analyzes source language: lexer, parser, semantic analyzer, and IR generation.

Expected points: source-language dependent.

Common mistake: including target-specific code generation.

10. What is compiler backend?

Answer: The part that optimizes and generates target code from IR.

Expected points: target-machine dependent.

Common mistake: saying backend parses source code.

7. Deep-Dive Questions

1. Why split compiler into phases?

It improves modularity, debugging, maintenance, and reuse. Different languages can share a backend, and different targets can share a frontend.

2. Can phases overlap?

Yes. Real compilers may combine or repeat phases for efficiency. For teaching, phases are separated conceptually.

3. What is a compiler pass?

A pass is one complete traversal over source, AST, or IR. A phase may contain multiple passes.

4. What is the difference between analysis and synthesis phases?

Analysis breaks source into structure and meaning. Synthesis constructs target code from that understanding.

5. Why is error handling difficult?

After one error, the compiler must recover enough to report more errors without producing misleading messages.

8. Comparison Tables

Aspect	Frontend	Backend
Focus	Source language	Target machine
Main work	Lexing, parsing, semantic checks	Optimization, code generation
Input	Source code	IR
Output	IR	Machine/bytecode
Example issue	Type mismatch	Register shortage

Aspect	Analysis Phase	Synthesis Phase
Meaning	Understand source	Produce output
Includes	Lexer, parser, semantic analyzer	IR generation, optimization, code generation
Direction	Source to meaning	Meaning to target code

9. Common Mistakes

- Confusing tokenization with parsing.
- Forgetting semantic analysis.
- Thinking optimization always happens after code generation.
- Thinking symbol table belongs to only one phase.
- Assuming all compilers use exactly the same phase order internally.

10. Edge Cases / Special Cases

- Some compilers compile directly to bytecode.
- Some interpreters still use compiler phases internally.
- Optimization can happen at IR level, machine level, or link time.
- Syntax can be valid while semantics are invalid.
- Error recovery is part of practical compiler design.

11. How to Explain in Interview

"A compiler converts source code to target code through phases: lexical analysis makes tokens, parsing checks grammar, semantic analysis checks meaning, IR generation creates machine-independent code, optimization improves it, and code generation emits target instructions. The symbol table and error handling support these phases."

12. Quick Revision Notes

- Lexer: characters to tokens.
- Parser: tokens to syntax structure.
- Semantic analyzer: types, scope, declarations.
- IR: machine-independent representation.
- Optimizer: improves without changing meaning.
- Code generator: target instructions.

13. Practice Tasks

1. Tokenize `int x = a + 5;`.
2. Draw a parse tree for `a + b * c`.
3. Find semantic errors in small C snippets.
4. Convert expressions to three-address code.
5. Identify compiler phases used by a SQL engine.

14. Final Cheat Sheet

Item	Answer
Core definition	Compiler phases are steps from source code to target code
Why it matters	Makes compilation modular and optimizable
Most asked questions	List phases, frontend/backend, symbol table
Common comparisons	Lexical vs syntax vs semantic analysis
One-line answer	Compiler phases progressively convert text into efficient executable code

Token vs Lexeme

1. Overview

Definition

A **lexeme** is the actual character sequence in source code.

A **token** is the category or class assigned to that lexeme by the lexer.

Example:

```
int count = 10;
```

Lexeme	Token
int	KEYWORD

Lexeme	Token
count	IDENTIFIER
=	ASSIGN_OPERATOR
10	NUMBER
;	SEMICOLON

Why It Matters

Parsers do not want to process raw characters. They work better with token categories.

Where It Is Used in Real Systems

Lexers in C compilers, Java compilers, TypeScript compilers, SQL engines, JSON parsers, code editors, syntax highlighters, and static analyzers use tokens and lexemes.

Why Interviewers Ask

This checks whether you understand the first phase of compilation. Many students memorize lexer definitions but confuse token, pattern, and lexeme.

2. Core Idea

Intuition

A lexeme is the exact word. A token is its role.

Real-World Analogy

In the sentence:

```
Ravi eats rice.
```

Word	Category
Ravi	Noun
eats	Verb
rice	Noun

Similarly:

```
sum = a + b;
```

Lexeme	Token
sum	IDENTIFIER
=	ASSIGN
a	IDENTIFIER
+	PLUS
b	IDENTIFIER
;	SEMI

Step-by-Step Explanation

1. Lexer reads characters.
2. It matches character sequences against token patterns.
3. It emits token objects.
4. Token may store lexeme and extra attributes.

Example token object:

```
Token(type=IDENTIFIER, lexeme="count", line=1)
```

3. Important Subtopics

Token

A token is an abstract category used by the parser.

Why it matters: grammar rules are written using token types.

Example: **IDENTIFIER**, **NUMBER**, **PLUS**.

Interview angle: token is not always the actual string.

Lexeme

A lexeme is the exact substring from source code.

Why it matters: compiler needs exact names and literal values.

Example: **count**, **42**, **while**.

Interview angle: multiple lexemes can belong to one token class.

Pattern

A pattern defines the rule for recognizing lexemes of a token.

Why it matters: lexers use regular expressions or automata.

Example: identifier pattern `[A-Za-z_][A-Za-z0-9_]*`.

Interview angle: token vs pattern vs lexeme.

Attribute

An attribute stores additional information with a token.

Why it matters: parser and semantic analyzer need values.

Example: NUMBER token may store numeric value `42`.

Interview angle: why token needs attributes.

4. Real-World Example

In a code editor, syntax highlighting uses tokenization. The editor recognizes `if` as a keyword token, `"hello"` as a string token, and `x` as an identifier token. It colors each category differently.

5. Diagrams / Mental Models

Source code:

```
int age = 20;
```

Characters -> Lexemes -> Tokens

"int" -> KEYWORD

"age" -> IDENTIFIER

"=" -> ASSIGN

"20" -> NUMBER

";" -> SEMICOLON

6. Common Interview Questions

1. What is a token?

Answer: A token is a category of lexemes recognized by the lexer.

Expected points: token type, parser input.

Common mistake: saying token is always the exact word.

2. What is a lexeme?

Answer: A lexeme is the actual sequence of characters matched in source code.

Expected points: actual text.

Common mistake: confusing it with token class.

3. Give an example of token and lexeme.

Answer: In `int x;`, `int` is a lexeme with token KEYWORD, and `x` is a lexeme with token IDENTIFIER.

Expected points: clear mapping.

Common mistake: calling `int` only a token.

4. What is a token pattern?

Answer: A pattern is the rule used to identify lexemes for a token.

Expected points: regex/automata.

Common mistake: saying pattern and lexeme are same.

5. Can many lexemes map to one token?

Answer: Yes. `x`, `sum`, and `total1` can all map to IDENTIFIER.

Expected points: category abstraction.

Common mistake: thinking one token has only one lexeme.

6. Can one lexeme belong to different tokens?

Answer: Usually tokenization rules choose one token based on context or priority. For example, `if` could match identifier pattern, but keyword priority classifies it as KEYWORD.

Expected points: priority, reserved words.

Common mistake: ignoring lexer rule priority.

7. Why does token store lexeme?

Answer: The compiler needs actual names and values for identifiers and literals.

Expected points: symbol table, semantic analysis.

Common mistake: thinking token type alone is enough.

8. Are whitespace and comments tokens?

Answer: Usually they are skipped, but some languages/tools may preserve them.

Expected points: depends on compiler/tool.

Common mistake: saying never.

9. What is an attribute value?

Answer: Extra information attached to a token, such as numeric value, symbol table pointer, or string content.

Expected points: useful for later phases.

Common mistake: treating all tokens as plain strings.

10. What is the parser input?

Answer: A stream of tokens produced by the lexical analyzer.

Expected points: parser consumes token types and attributes.

Common mistake: saying parser reads raw characters directly.

7. Deep-Dive Questions

1. Why are keywords checked before identifiers?

Because keywords like `if` also match identifier patterns. Lexer priority or reserved-word lookup ensures they become keyword tokens.

2. How is `==` handled differently from `=`?

The lexer usually uses longest match, so `==` becomes equality token instead of two assignment tokens.

3. Why does a token need line number?

For accurate error reporting and debugging.

4. How are numeric literals handled?

The lexer recognizes the lexeme and may convert it into an internal numeric value stored as an attribute.

5. Can comments be preserved?

Yes, formatters, documentation generators, and IDE tools may preserve comments, though compilers often skip them.

8. Comparison Tables

Aspect	Token	Lexeme
Meaning	Category/class	Actual text
Example	IDENTIFIER	<code>total</code>
Used by	Parser	Lexer and semantic analysis
Level	Abstract	Concrete
Many-to-one?	One token class can represent many lexemes	Many lexemes can map to same token

Aspect	Token	Pattern
Meaning	Recognized category	Rule to recognize category
Example	NUMBER	<code>[0-9]+</code>
Role	Parser input	Lexer specification

9. Common Mistakes

- Saying token and lexeme are synonyms.
- Forgetting token attributes.
- Ignoring keyword priority.
- Thinking parser directly reads characters.
- Confusing pattern with token.

10. Edge Cases / Special Cases

- `if` may match identifier regex but is classified as keyword.
- `>=` must be recognized before `>`.
- Whitespace may matter in Python but not in C.
- String literals may contain escaped characters.
- Some tokens depend on lexical modes, such as inside comments or strings.

11. How to Explain in Interview

"A lexeme is the actual text found in the source program, while a token is the category assigned to it. For example, in `count = 10`, `count` is a lexeme and IDENTIFIER is its token. The parser consumes tokens, not raw characters."

12. Quick Revision Notes

- Lexeme: actual substring.
- Token: category.
- Pattern: matching rule.
- Attribute: extra token data.
- Trap: keywords can match identifier patterns.

13. Practice Tasks

1. List tokens and lexemes for `while (x <= 10) x++;`.
2. Write regex for identifiers and integers.
3. Explain why `==` must use longest match.
4. Classify keywords vs identifiers.
5. Build a tiny tokenizer for arithmetic expressions.

14. Final Cheat Sheet

Item	Answer
Core definition	Lexeme is actual text; token is its category
Why it matters	Parser uses token stream
Most asked questions	token vs lexeme vs pattern
Common comparison	Token vs Lexeme
One-line answer	Lexeme is what appears; token is what it means to the lexer

Lexical Analysis vs Parsing

1. Overview

Definition

Lexical analysis converts raw characters into tokens.

Parsing converts a token stream into a grammatical structure such as a parse tree or AST.

Why It Matters

These are the first two major compiler phases. If you mix them up, your understanding of compiler design becomes weak.

Where It Is Used in Real Systems

Lexing and parsing are used in compilers, interpreters, SQL engines, HTML parsers, JSON parsers, template engines, linters, and IDEs.

Why Interviewers Ask

Interviewers ask this to see if you understand separation of concerns:

```
Characters -> Tokens -> Syntax Tree
```

2. Core Idea

Intuition

Lexical analysis identifies words. Parsing checks sentence grammar.

Real-World Analogy

For English:

```
The boy eats rice.
```

Lexical analysis identifies words: **The**, **boy**, **eats**, **rice**.

Parsing checks structure: subject + verb + object.

Small Example

Expression:

```
a + b * c
```

Lexer output:

```
ID PLUS ID MUL ID
```

Parser structure:



This shows `*` has higher precedence than `+`.

Step-by-Step Explanation

1. Lexer reads characters.
2. Lexer removes whitespace/comments where appropriate.
3. Lexer emits tokens.
4. Parser reads tokens.
5. Parser checks grammar rules.
6. Parser builds parse tree or AST.

3. Important Subtopics

Lexical Analyzer

Recognizes identifiers, keywords, numbers, operators, separators, and literals.

Why it matters: simplifies parser input.

Example: `x = 5;` becomes `ID ASSIGN NUM SEMI`.

Interview angle: regex and DFA use.

Parser

Checks token order using grammar.

Why it matters: validates program structure.

Example: `if (x) y = 1;` fits an if-statement grammar.

Interview angle: LL vs LR.

Syntax Error

An error where tokens do not follow grammar.

Why it matters: parser detects it.

Example: `x = ;`.

Interview angle: error recovery.

Lexical Error

An invalid character sequence.

Why it matters: lexer detects it.

Example: illegal character `@` in some languages.

Interview angle: lexer vs parser responsibility.

4. Real-World Example

In a JSON parser:

```
{"age": 20}
```

Lexer identifies `{`, string `"age"`, `:`, number `20`, `}`.

Parser checks JSON grammar: object contains key-value pairs.

5. Diagrams / Mental Models

```
Raw text:
if (x > 0) y = x;

      |
      v

Lexical analysis:
IF LPAREN ID GT NUM RPAREN ID ASSIGN ID SEMI

      |
      v

Parsing:
IfStatement(condition, assignment)
```

6. Common Interview Questions

1. What is lexical analysis?

Answer: It converts character streams into tokens.

Expected points: tokenization, removes whitespace/comments.

Common mistake: saying it builds parse tree.

2. What is parsing?

Answer: It checks token sequence against grammar and builds syntax structure.

Expected points: grammar, parse tree/AST.

Common mistake: saying it reads raw characters first.

3. Difference between lexical analysis and parsing?

Answer: Lexing handles characters and token patterns; parsing handles token sequences and grammar.

Expected points: characters vs tokens.

Common mistake: using vague "both check syntax."

4. Who detects invalid variable names?

Answer: Usually the lexer detects invalid token patterns.

Expected points: identifier regex.

Common mistake: always saying parser.

5. Who detects missing semicolon?

Answer: Parser usually detects it because grammar expects a semicolon token.

Expected points: grammar structure.

Common mistake: saying lexer.

6. Why separate lexer and parser?

Answer: It simplifies compiler design and lets parser work with meaningful symbols.

Expected points: modularity.

Common mistake: saying separation is only for speed.

7. Can parsing happen without lexing?

Answer: Some scannerless parsers parse characters directly, but traditional compilers separate lexing and parsing.

Expected points: special case.

Common mistake: saying impossible.

8. Does lexer understand precedence?

Answer: No. Operator precedence is handled by parser/grammar, not lexer.

Expected points: parser responsibility.

Common mistake: lexer builds expression tree.

9. What does parser output?

Answer: Parse tree, AST, or syntax structure.

Expected points: parse tree vs AST.

Common mistake: saying tokens.

10. What does lexer output?

Answer: Token stream.

Expected points: token type and attributes.

Common mistake: saying machine code.

7. Deep-Dive Questions

1. Why are regular expressions enough for lexing but not parsing?

Most tokens are regular patterns, but programming language syntax often needs nested structure, which requires context-free grammars.

2. What is scannerless parsing?

It combines lexical and syntax analysis by parsing raw characters directly.

3. Can whitespace affect parsing?

Yes. In Python indentation affects block structure, so the lexer emits INDENT/DEDENT tokens.

4. How does parser handle precedence?

Using grammar structure, precedence declarations, or parser algorithms.

5. What is lexical mode?

A lexer state used when token rules change, such as inside a string, comment, or template literal.

8. Comparison Tables

Aspect	Lexical Analysis	Parsing
Input	Characters	Tokens

Aspect	Lexical Analysis	Parsing
Output	Tokens	Parse tree/AST
Rules	Regular expressions/automata	Context-free grammar
Detects	Invalid characters/tokens	Invalid syntax structure
Example error	<code>12abc</code> if illegal	<code>x = ;</code>

9. Common Mistakes

- Saying lexer handles grammar.
- Saying parser handles character-level matching.
- Thinking precedence is a lexical concept.
- Forgetting lexer may skip comments.
- Ignoring indentation-sensitive languages.

10. Edge Cases / Special Cases

- Python uses indentation tokens.
- Template literals may require lexer modes.
- C++ tokenization has context-sensitive complications.
- Scannerless parsers combine both phases.
- Some tools preserve comments and whitespace.

11. How to Explain in Interview

"Lexical analysis converts source characters into tokens like identifiers and numbers. Parsing takes those tokens and checks whether they follow grammar rules, producing a parse tree or AST. Lexing is about words; parsing is about sentence structure."

12. Quick Revision Notes

- Lexer input: characters.
- Lexer output: tokens.
- Parser input: tokens.
- Parser output: parse tree/AST.
- Regex for lexing; CFG for parsing.

13. Practice Tasks

1. Tokenize `if (a >= 10) b = a;`.
2. Draw parse tree for `a + b * c`.
3. Identify lexer errors and parser errors in examples.
4. Write a regex for numbers.
5. Write a grammar for arithmetic expressions.

14. Final Cheat Sheet

Item	Answer
Core definition	Lexing makes tokens; parsing checks grammar
Why it matters	Separates character recognition from syntax structure
Most asked questions	lexer output, parser output, regex vs CFG
Common comparison	Lexical analysis vs parsing
One-line answer	Lexer finds words; parser checks grammar

Why Regular Expressions Are Used in Lexers

1. Overview

Definition

Regular expressions are formal patterns used to describe tokens such as identifiers, numbers, keywords, operators, and whitespace.

Why It Matters

Most lexical tokens have regular structure. Regular expressions can be converted into finite automata, making token recognition efficient and systematic.

Where It Is Used in Real Systems

Lexers in compilers, syntax highlighters, search tools, data validators, log parsers, and command-line tools use regex-like rules.

Why Interviewers Ask

This checks whether you understand why lexers use regular languages and why parsers need stronger grammar models.

2. Core Idea

Intuition

A token usually follows a simple pattern.

Examples:

Token	Pattern
Integer	<code>[0-9]+</code>
Identifier	<code>[A-Za-z_][A-Za-z0-9_]*</code>
Whitespace	<code>[\t\n]+</code>

Real-World Analogy

A form validator checks if an email has a valid pattern. Similarly, a lexer checks if a sequence of characters matches a token pattern.

Small Example

For:

```
count123 = 45;
```

Regex rules:

```
IDENTIFIER: [A-Za-z_][A-Za-z0-9_]*  
NUMBER:    [0-9]+  
ASSIGN:    =  
SEMI:      ;
```

Output:

```
IDENTIFIER(count123), ASSIGN, NUMBER(45), SEMI
```

Step-by-Step Explanation

1. Define regex for each token.
2. Combine regex rules.
3. Convert regex to NFA.
4. Convert NFA to DFA.
5. Use DFA to scan input efficiently.

3. Important Subtopics

Regular Language

A regular language can be recognized by a finite automaton.

Why it matters: tokens are usually regular.

Example: all valid identifiers.

Interview angle: regex vs CFG.

Finite Automata

Automata recognize token patterns by changing states.

Why it matters: efficient implementation.

Example: digit loop for numbers.

Interview angle: regex to NFA/DFA.

Longest Match

Lexer chooses the longest valid lexeme.

Why it matters: `>=` should not become `>` and `=`.

Example: `==` becomes EQ, not ASSIGN ASSIGN.

Interview angle: maximal munch.

Rule Priority

If two rules match the same lexeme, priority decides.

Why it matters: `if` should be keyword, not identifier.

Example: keyword rule before identifier rule.

Interview angle: reserved words.

4. Real-World Example

A SQL lexer recognizes:

```
SELECT name FROM users WHERE age >= 18;
```

Regex rules identify keywords, identifiers, numbers, comparison operators, and semicolons before the SQL parser builds a query tree.

5. Diagrams / Mental Models

```
Regex rules
  |
  v
NFA
  |
  v
DFA
  |
  v
Fast token scanner
```

Identifier DFA:

```
Start --letter/_--> InIdentifier --letter/digit/_--> InIdentifier
```

6. Common Interview Questions

1. Why are regex used in lexical analysis?

Answer: Because tokens usually form regular languages, which regex describe compactly and efficiently.

Expected points: token patterns, automata.

Common mistake: saying regex can parse full programming languages.

2. Can regex parse nested expressions?

Answer: Regular expressions cannot generally parse nested recursive structures; parsers use context-free grammars.

Expected points: regular vs context-free.

Common mistake: saying regex can parse everything.

3. What is maximal munch?

Answer: The lexer chooses the longest possible valid token.

Expected points: `>=`, `==`, identifiers.

Common mistake: choosing first character match.

4. How are keywords recognized?

Answer: Either by separate keyword rules with priority or by checking identifier lexemes against a keyword table.

Expected points: priority/reserved lookup.

Common mistake: ignoring that keywords match identifier regex.

5. What is regex to DFA conversion?

Answer: Regex can be converted to NFA and then DFA for efficient scanning.

Expected points: finite automata.

Common mistake: thinking runtime regex engine is always used directly.

6. Why use DFA for lexers?

Answer: DFA scans input in linear time with deterministic transitions.

Expected points: speed, deterministic state.

Common mistake: saying DFA stores parse tree.

7. Are comments recognized by regex?

Answer: Many simple comments can be recognized lexically, but nested comments may need more complex handling.

Expected points: special cases.

Common mistake: saying always trivial.

8. What token rules need priority?

Answer: Keywords vs identifiers, multi-character operators vs single-character operators.

Expected points: ambiguity resolution.

Common mistake: no priority needed.

9. Why not use CFG for lexing?

Answer: Regex/automata are simpler and faster for token-level patterns.

Expected points: right tool for simple patterns.

Common mistake: using more powerful tools unnecessarily.

10. What is a lexical specification?

Answer: A list of token names and regex patterns used to build a lexer.

Expected points: lex/flex style rules.

Common mistake: confusing it with grammar.

7. Deep-Dive Questions

1. Why are regular languages less powerful than context-free languages?

Regular languages cannot count arbitrary nested structures because finite automata have finite memory.

2. How does lexer handle ambiguity?

Usually by longest match first, then rule priority.

3. Why are DFAs efficient?

At each character, there is exactly one next state, so scanning is linear.

4. Can lexer require context?

Sometimes yes. Languages like C++ and JavaScript have context-sensitive lexical cases.

5. Why do lexer generators exist?

They convert regex token specifications into efficient scanner code automatically.

8. Comparison Tables

Aspect	Regular Expression	Context-Free Grammar
Used for	Token patterns	Program syntax
Recognizes	Regular languages	Context-free languages
Memory model	Finite automata	Pushdown automata
Good for	Identifiers, numbers	Nested expressions, statements
Example	<code>[0-9]+</code>	<code>E -> E + T</code>

9. Common Mistakes

- Saying regex can parse nested blocks.
- Forgetting longest match.
- Forgetting keyword priority.
- Confusing regex tokens with grammar productions.
- Thinking lexers always use backtracking regex engines.

10. Edge Cases / Special Cases

- Nested comments may not be regular.
- String literals need escape handling.
- Some languages have context-sensitive tokenization.
- Keywords may be contextual, such as `async` in some languages.
- Whitespace can be meaningful.

11. How to Explain in Interview

"Regular expressions are used in lexers because most tokens, like identifiers and numbers, follow regular patterns. These regex rules can be converted to finite automata, allowing fast linear scanning. But regex is not enough for nested syntax, so parsing uses grammars."

12. Quick Revision Notes

- Regex describes token patterns.
- Regex -> NFA -> DFA.
- DFA scanning is fast.
- Longest match matters.
- Regex cannot handle general nested syntax.

13. Practice Tasks

1. Write regex for identifiers.
2. Write regex for integer and floating literals.
3. Trace tokenization of `a >= b`.

4. Build a simple DFA for numbers.
5. Explain why parentheses nesting is not regular.

14. Final Cheat Sheet

Item	Answer
Core definition	Regex describes token patterns
Why it matters	Enables efficient lexer construction
Most asked questions	regex vs CFG, DFA, maximal munch
Common comparison	Regular expression vs grammar
One-line answer	Regex is perfect for token patterns, not full program structure

LL vs LR Parsers

1. Overview

Definition

LL parsers read input from Left to right and produce a Leftmost derivation.

LR parsers read input from Left to right and produce a Rightmost derivation in reverse.

Why It Matters

LL and LR are two major parsing strategies. They differ in power, implementation style, grammar restrictions, and error detection.

Where It Is Used in Real Systems

Parser Type	Used In
LL / recursive descent	Hand-written parsers, simple languages, config formats
LR / LALR	Yacc/Bison, many production compilers

Why Interviewers Ask

This tests whether you understand top-down vs bottom-up parsing.

2. Core Idea

Intuition

LL parsing predicts what grammar rule to use before reading the full construct.

LR parsing waits, reads input, and reduces recognized pieces into larger structures.

Real-World Analogy

LL parser: You assemble furniture by predicting the next part from the manual.

LR parser: You collect parts first and then combine them when they match a known structure.

Small Example

Grammar:

```
E -> T E'
E' -> + T E' | epsilon
T -> id
```

LL parser can predict using FIRST sets.

For LR, stack operations happen:

```
Shift id
Reduce id to T
Shift +
Shift id
Reduce id to T
Reduce T + T to E
```

Step-by-Step Explanation

LL:

1. Start from start symbol.
2. Look at current input token.
3. Choose production.
4. Expand nonterminal.
5. Match terminals.

LR:

1. Start with empty stack.
2. Shift tokens onto stack.
3. Reduce stack symbols when RHS of production appears.
4. Accept when start symbol is produced.

3. Important Subtopics

Top-Down Parsing

Builds parse tree from root to leaves.

Why it matters: easier to implement manually.

Example: recursive descent parser.

Interview angle: LL parser.

Bottom-Up Parsing

Builds parse tree from leaves to root.

Why it matters: handles more grammars.

Example: LR parser.

Interview angle: shift-reduce parsing.

Lookahead

Number of input tokens used to decide parsing action.

Why it matters: resolves choices.

Example: LL(1), LR(1).

Interview angle: what does (1) mean?

Shift-Reduce Parsing

LR parsers shift input tokens and reduce when handles are found.

Why it matters: core LR mechanism.

Example: reduce `id` to expression.

Interview angle: shift-reduce conflict.

Grammar Restrictions

LL grammars cannot handle left recursion directly. LR parsers can handle many left-recursive grammars.

Why it matters: grammar design.

Example: `E -> E + T`.

Interview angle: left recursion removal.

4. Real-World Example

A hand-written parser for a small programming language is often recursive descent, which is LL-style. A production compiler generated using Bison is often LALR, an LR-family parser.

5. Diagrams / Mental Models

LL Parsing:

Start Symbol

|

v

Expand grammar rules

|

v

Match input tokens

LR Parsing:

Input tokens -> Shift onto stack -> Reduce handles -> Start Symbol

6. Common Interview Questions

1. What does LL mean?

Answer: Left-to-right input scan and leftmost derivation.

Expected points: top-down parsing.

Common mistake: confusing second L with rightmost derivation.

2. What does LR mean?

Answer: Left-to-right input scan and rightmost derivation in reverse.

Expected points: bottom-up parsing.

Common mistake: saying LR scans right to left.

3. Difference between LL and LR?

Answer: LL is top-down and predictive; LR is bottom-up and shift-reduce.

Expected points: derivation, power, left recursion.

Common mistake: only saying one is faster.

4. Which parser is more powerful?

Answer: LR parsers handle a larger class of grammars than LL parsers.

Expected points: grammar coverage.

Common mistake: saying LL is always better because simpler.

5. Can LL parser handle left recursion?

Answer: No, direct left recursion causes infinite recursion in top-down parsing.

Expected points: remove left recursion.

Common mistake: saying all parsers reject left recursion.

6. Can LR parser handle left recursion?

Answer: Yes, LR parsers can naturally handle many left-recursive grammars.

Expected points: bottom-up parsing.

Common mistake: removing left recursion unnecessarily for LR.

7. What is LL(1)?

Answer: An LL parser using one token of lookahead.

Expected points: FIRST/FOLLOW parsing table.

Common mistake: saying one production only.

8. What is LR(1)?

Answer: An LR parser using one lookahead token to decide shift/reduce actions.

Expected points: LR items.

Common mistake: saying it reads one token total.

9. What is shift-reduce conflict?

Answer: Parser cannot decide whether to shift next token or reduce current stack symbols.

Expected points: ambiguity or precedence issues.

Common mistake: calling it lexer error.

10. Which parser is easier to implement manually?

Answer: LL recursive descent parsers are usually easier to hand-write.

Expected points: readability and simple control flow.

Common mistake: saying LR is always hand-written.

7. Deep-Dive Questions

1. Why is LR more powerful than LL?

LR parsers delay decisions until enough input has been seen, while LL parsers must predict productions early.

2. What is LALR?

LALR is a practical LR variant that merges similar parser states to reduce table size.

3. What causes reduce-reduce conflict?

Two different reductions are possible for the same parser state and lookahead.

4. Why are recursive descent parsers popular?

They are simple, readable, debuggable, and easy to customize.

5. How does precedence solve conflicts?

Parser generators can use precedence and associativity declarations to choose shift or reduce.

8. Comparison Tables

Aspect	LL Parser	LR Parser
Direction	Top-down	Bottom-up
Derivation	Leftmost	Rightmost in reverse
Mechanism	Predict/expand	Shift/reduce
Handles left recursion	No	Yes
Grammar power	Less powerful	More powerful
Manual implementation	Easier	Harder
Tools	Recursive descent, ANTLR	Yacc, Bison

9. Common Mistakes

- Saying LR scans right to left.
- Forgetting LR derives rightmost derivation in reverse.
- Thinking LL and LR both cannot handle left recursion.
- Confusing lookahead with total input length.
- Thinking parser type determines language speed.

10. Edge Cases / Special Cases

- Some recursive descent parsers use Pratt parsing for expressions.
- LL grammar may need left factoring.
- LR parser tables may have conflicts for ambiguous grammars.
- LALR is common because canonical LR(1) tables can be large.
- PEG parsers are another parsing model.

11. How to Explain in Interview

"LL parsers are top-down parsers that predict productions and produce leftmost derivations. LR parsers are bottom-up shift-reduce parsers that produce rightmost derivations in reverse. LL is easier to implement manually, while LR handles more grammars, including left-recursive ones."

12. Quick Revision Notes

- LL: top-down, leftmost derivation.
- LR: bottom-up, rightmost derivation in reverse.
- LL cannot handle left recursion.
- LR uses shift/reduce.
- (1) means one lookahead token.

13. Practice Tasks

1. Trace LL parsing for `id + id`.
2. Trace LR shift-reduce parsing for `id + id`.
3. Remove left recursion from a grammar.
4. Left-factor a grammar.
5. Build FIRST/FOLLOW table for LL(1).

14. Final Cheat Sheet

Item	Answer
Core definition	LL predicts top-down; LR reduces bottom-up
Why it matters	Parser choice affects grammar design
Most asked questions	LL(1), LR(1), left recursion, conflicts
Common comparison	LL vs LR
One-line answer	LL expands from start symbol; LR reduces input back to start symbol

Left Recursion and Left Factoring

1. Overview

Definition

Left recursion occurs when a nonterminal can derive a string beginning with itself.

Direct left recursion:

```
A -> A alpha | beta
```

Left factoring rewrites grammar rules that share common prefixes so a top-down parser can choose productions with limited lookahead.

Example:

```
A -> alpha beta | alpha gamma
```

becomes:

```
A -> alpha A'  
A' -> beta | gamma
```

Why It Matters

LL parsers cannot handle left recursion and struggle with common prefixes. Removing left recursion and applying left factoring helps build predictive parsers.

Where It Is Used in Real Systems

Parser design, compiler construction labs, recursive descent parsers, grammar engineering, and parser generator preprocessing.

Why Interviewers Ask

This tests grammar transformation skills, a common compiler-design placement question.

2. Core Idea

Intuition

Left recursion causes top-down parsers to call the same function before consuming input.

Left factoring handles delayed decision-making.

Real-World Analogy

Left recursion is like saying, "To solve problem A, first solve problem A." You never progress.

Left factoring is like two roads starting with the same path. Walk the common path first, then choose the branch.

Small Example

Left-recursive grammar:

```
E -> E + T | T
```

Remove left recursion:

```
E -> T E'
E' -> + T E' | epsilon
```

Left factoring:

```
S -> if E then S else S | if E then S
```

Factor:

```
S -> if E then S S'
S' -> else S | epsilon
```

Step-by-Step Explanation

For direct left recursion:

```
A -> A alpha | beta
```

Rewrite as:

```
A -> beta A'
A' -> alpha A' | epsilon
```

This makes the grammar suitable for top-down parsing.

3. Important Subtopics

Direct Left Recursion

A rule immediately calls itself on the left.

Why it matters: causes infinite recursion in recursive descent.

Example: $A \rightarrow A a \mid b$.

Interview angle: remove it.

Indirect Left Recursion

A nonterminal reaches itself through other nonterminals.

Example:

```
A -> B a
B -> A b | c
```

Why it matters: harder to detect.

Interview angle: identify and remove.

Left Factoring

Extracts common prefixes.

Why it matters: makes predictive parsing possible.

Example: $A \rightarrow ab \mid ac$ becomes $A \rightarrow a A'$.

Interview angle: LL(1) grammar transformation.

Epsilon Productions

Rules deriving empty string.

Why it matters: often introduced during transformations.

Example: $A' \rightarrow \alpha A' \mid \epsilon$.

Interview angle: FIRST/FOLLOW interaction.

4. Real-World Example

In a hand-written expression parser, left-recursive grammar like:

```
Expr -> Expr + Term | Term
```

cannot be directly implemented as:

```
def expr():
    expr()
    match("+")
    term()
```

It would recurse forever. The grammar must be rewritten or handled using another parsing strategy.

5. Diagrams / Mental Models

Left recursion:

```

E
|
E + T
|
E + T + T
|
...

```

Left factoring:

```

Before:
S -> common A
S -> common B

After:
S -> common S'
S' -> A | B

```

6. Common Interview Questions

1. What is left recursion?

Answer: A grammar is left-recursive if a nonterminal can derive a sentential form starting with itself.

Expected points: direct and indirect.

Common mistake: only knowing direct left recursion.

2. Why is left recursion a problem for LL parsers?

Answer: It causes infinite recursion because the parser expands the same nonterminal without consuming input.

Expected points: top-down parsing.

Common mistake: saying it affects all parsers.

3. Remove left recursion from $E \rightarrow E + T \mid T$.

Answer:

```

E -> T E'
E' -> + T E' | epsilon

```

Expected points: preserve language.

Common mistake: losing repetition.

4. What is left factoring?

Answer: Rewriting grammar to factor out common prefixes.

Expected points: predictive parsing.

Common mistake: confusing with left recursion removal.

5. Left-factor $A \rightarrow ab \mid ac$.

Answer:

```
A  -> a A'
A' -> b | c
```

Expected points: common prefix.

Common mistake: changing language.

6. Is left recursion bad for LR parsers?

Answer: No. LR parsers can often handle left-recursive grammars naturally.

Expected points: parser-specific issue.

Common mistake: removing it always.

7. What is indirect left recursion?

Answer: A nonterminal derives itself through one or more other nonterminals.

Expected points: chain derivation.

Common mistake: only checking immediate rules.

8. Does left factoring remove ambiguity?

Answer: Not necessarily. It removes common-prefix prediction problems, not all ambiguity.

Expected points: distinction.

Common mistake: saying yes always.

9. Why does left factoring help LL(1)?

Answer: It delays production choice until enough input is consumed.

Expected points: one-token lookahead.

Common mistake: saying it changes precedence.

10. Can transformations affect parse tree shape?

Answer: Yes, parse tree shape may change, but language should remain the same.

Expected points: grammar equivalence.

Common mistake: assuming parse tree always identical.

7. Deep-Dive Questions

1. How do you remove indirect left recursion?

Order nonterminals, substitute earlier productions into later ones, then eliminate direct left recursion.

2. Does removing left recursion change associativity?

It can affect parse tree shape. Semantic actions must be adjusted to preserve intended associativity.

3. Why does expression grammar often use left recursion?

Left recursion naturally represents left associativity, such as $a - b - c$.

4. Can left factoring create epsilon rules?

Yes. If one alternative is just the common prefix, epsilon may be needed.

5. Is every grammar convertible to LL(1)?

No. Some languages/grammars cannot be made LL(1) without changing parsing strategy or grammar design.

8. Comparison Tables

Aspect	Left Recursion	Left Factoring
Problem	Recursive call before consuming input	Common prefix creates prediction conflict
Affects	LL/top-down parsers	LL/predictive parsers
Example	$A \rightarrow A a$	b
Fix	$A \rightarrow b A'$	$A \rightarrow a A'$
Goal	Prevent infinite recursion	Delay decision

9. Common Mistakes

- Confusing left recursion with left factoring.
- Thinking left recursion affects LR parsers the same way.
- Removing left recursion incorrectly and changing language.
- Forgetting epsilon.
- Thinking left factoring removes all ambiguity.

10. Edge Cases / Special Cases

- Indirect left recursion is less obvious.

- Epsilon productions complicate FIRST/FOLLOW.
- Left recursion may be preferred in LR grammars.
- Removing left recursion can affect semantic action placement.
- Dangling else may remain ambiguous even after factoring.

11. How to Explain in Interview

"Left recursion occurs when a nonterminal can derive itself as the leftmost symbol, which causes infinite recursion in top-down parsers. We remove it by introducing a helper nonterminal. Left factoring extracts common prefixes so an LL parser can delay choices until enough input is seen."

12. Quick Revision Notes

- Left recursion: $A \rightarrow A \alpha \mid \beta$.
- Remove as $A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \epsilon$.
- Left factoring: extract common prefix.
- LL parsers need both transformations.
- LR parsers can handle left recursion.

13. Practice Tasks

1. Remove left recursion from $A \rightarrow A a \mid b$.
2. Left-factor $S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S$.
3. Identify indirect left recursion.
4. Check whether transformed grammar preserves strings.
5. Build LL(1) table after transformations.

14. Final Cheat Sheet

Item	Answer
Core definition	Left recursion repeats from left; left factoring extracts common prefix
Why it matters	Required for predictive parsing
Most asked questions	remove left recursion, factor grammar
Common comparison	Left recursion vs left factoring
One-line answer	Remove left recursion to avoid infinite top-down parsing; left-factor to make choices predictable

FIRST and FOLLOW

1. Overview

Definition

FIRST(X) is the set of terminals that can appear first in strings derived from X.

FOLLOW(A) is the set of terminals that can appear immediately after nonterminal A in some sentential form.

Why It Matters

FIRST and FOLLOW are used to build LL(1) parsing tables and decide which production to apply.

Where It Is Used in Real Systems

Parser generators, compiler courses, syntax analyzers, grammar validation tools, and recursive descent parser design.

Why Interviewers Ask

FIRST/FOLLOW questions test grammar reasoning and predictive parsing fundamentals.

2. Core Idea

Intuition

FIRST asks: "What can this begin with?"

FOLLOW asks: "What can come after this?"

Real-World Analogy

In a sentence pattern:

```
Greeting -> Hello Name | Hi Name
```

$\text{FIRST}(\text{Greeting}) = \{\text{Hello}, \text{Hi}\}.$

If:

```
Message -> Greeting !
```

$\text{FOLLOW}(\text{Greeting}) = \{!\}.$

Small Example

Grammar:

```
S -> A b  
A -> a | epsilon
```

$\text{FIRST}(A) = \{a, \text{epsilon}\}$

$\text{FIRST}(S) = \{a, b\}$ because if A disappears, b starts the string.

$\text{FOLLOW}(A) = \{b\}$ because b appears after A in $S \rightarrow A b$.

Step-by-Step Explanation

To compute FIRST:

1. $\text{FIRST}(\text{terminal})$ is the terminal itself.
2. If $A \rightarrow \text{epsilon}$, add epsilon to $\text{FIRST}(A)$.
3. If $A \rightarrow X Y$, add $\text{FIRST}(X)$ except epsilon.
4. If X can be epsilon, consider $\text{FIRST}(Y)$.

To compute FOLLOW:

1. Put $\$$ in $\text{FOLLOW}(\text{start symbol})$.
2. For $A \rightarrow \alpha B \beta$, add $\text{FIRST}(\beta)$ except epsilon to $\text{FOLLOW}(B)$.
3. If β can be epsilon, add $\text{FOLLOW}(A)$ to $\text{FOLLOW}(B)$.

3. Important Subtopics

FIRST of Terminal

FIRST of a terminal is itself.

Why it matters: base case.

Example: $\text{FIRST}(+) = \{+\}$.

Interview angle: avoid overthinking terminals.

FIRST of Nonterminal

Terminals that can begin derivations from that nonterminal.

Why it matters: production selection.

Example: $A \rightarrow aB \mid cD$, $\text{FIRST}(A) = \{a, c\}$.

Interview angle: epsilon handling.

FOLLOW of Start Symbol

Contains end marker $\$$.

Why it matters: parser knows when input can end.

Example: $\text{FOLLOW}(S)$ includes $\$$.

Interview angle: always include $\$$.

Epsilon Handling

If a symbol can vanish, next symbol matters.

Why it matters: most mistakes happen here.

Example: $A \rightarrow B C$, if B derives epsilon, $\text{FIRST}(C)$ contributes.

Interview angle: nullable symbols.

4. Real-World Example

In an LL(1) parser for expressions, FIRST helps choose whether to parse a term, factor, or operator continuation. FOLLOW helps decide when an epsilon production should be used.

5. Diagrams / Mental Models

```

FIRST(A):
A => ? ...
    ^
    first terminal
  
```

```

FOLLOW(A):
S => ... A ? ...
           ^
           terminal after A
  
```

LL(1) table idea:

```

Production A -> alpha goes into table[A, terminal] for each terminal in
FIRST(alpha).
If alpha can be epsilon, use FOLLOW(A).
  
```

6. Common Interview Questions

1. What is FIRST?

Answer: $\text{FIRST}(X)$ is the set of terminals that can begin strings derived from X.

Expected points: include epsilon if nullable.

Common mistake: only looking at first production symbol.

2. What is FOLLOW?

Answer: $\text{FOLLOW}(A)$ is the set of terminals that can appear immediately after nonterminal A.

Expected points: sentential forms.

Common mistake: using symbols before A.

3. Why is $\$$ added to FOLLOW(start)?

Answer: It marks end of input after the start symbol.

Expected points: accept condition.

Common mistake: forgetting $\$$.

4. How is epsilon handled in FIRST?

Answer: If a symbol can derive epsilon, continue checking the next symbol.

Expected points: nullable chain.

Common mistake: stopping too early.

5. How is epsilon handled in FOLLOW?

Answer: If beta after B can derive epsilon in $A \rightarrow \alpha B \beta$, add FOLLOW(A) to FOLLOW(B).

Expected points: propagation.

Common mistake: adding epsilon to FOLLOW.

6. Does FOLLOW contain epsilon?

Answer: No. FOLLOW contains terminals and $\$$, not epsilon.

Expected points: important trap.

Common mistake: adding epsilon.

7. Why are FIRST and FOLLOW used?

Answer: To construct predictive parsing tables and choose productions.

Expected points: LL(1).

Common mistake: saying lexer uses FOLLOW.

8. What is LL(1) condition using FIRST/FOLLOW?

Answer: Alternatives must have disjoint FIRST sets, and if epsilon exists, FIRST of other alternatives must not conflict with FOLLOW.

Expected points: no table conflicts.

Common mistake: only checking FIRST.

9. Compute FIRST of $A \rightarrow B C$ if B has epsilon.

Answer: FIRST(A) includes FIRST(B) except epsilon and also FIRST(C).

Expected points: nullable handling.

Common mistake: ignoring C.

10. Compute FOLLOW in $S \rightarrow A b$.

Answer: b is in FOLLOW(A).

Expected points: symbol after nonterminal.

Common mistake: adding FIRST(A).

7. Deep-Dive Questions

1. Can FIRST contain epsilon?

Yes, if the symbol or sequence can derive empty string.

2. Can FOLLOW contain epsilon?

No. FOLLOW represents actual terminals that may appear after a nonterminal.

3. What is FIRST of a sequence?

It is computed from left to right, considering nullable symbols.

4. How do FIRST/FOLLOW detect LL(1) conflicts?

If two productions compete for the same table cell, the grammar is not LL(1).

5. Why are computations iterative?

Because grammar rules depend on each other recursively, so sets grow until they stop changing.

8. Comparison Tables

Aspect	FIRST	FOLLOW
Meaning	What can start derivation	What can appear after nonterminal
Applies to	Terminals, nonterminals, sequences	Nonterminals
Can contain epsilon	Yes	No
Includes $\$$	Usually no	Start symbol yes
Used for	Choosing productions	Epsilon production decisions

9. Common Mistakes

- Adding epsilon to FOLLOW.
- Forgetting $\$$ in FOLLOW(start).
- Ignoring nullable symbols.
- Computing FIRST only from first rule.
- Confusing FIRST(A) with FOLLOW(A).

10. Edge Cases / Special Cases

- Chains like $A \rightarrow B C D$ where B and C are nullable.
- Recursive grammars require iterative fixed-point computation.
- Epsilon alternatives need FOLLOW for parsing table entries.
- Left recursion may complicate manual computation but sets still exist.
- Terminals have FIRST equal to themselves.

11. How to Explain in Interview

"FIRST tells what terminals can begin strings derived from a grammar symbol. FOLLOW tells what terminals can appear immediately after a nonterminal. They are mainly used to construct LL(1) parsing tables, especially to decide production choices and epsilon rules."

12. Quick Revision Notes

- FIRST may contain epsilon.
- FOLLOW never contains epsilon.
- FOLLOW(start) contains \$.
- Nullable symbols require looking ahead in the production.
- FIRST/FOLLOW build LL(1) parsing tables.

13. Practice Tasks

1. Compute FIRST/FOLLOW for $S \rightarrow A b, A \rightarrow a \mid \epsilon$.
2. Build LL(1) table for a simple grammar.
3. Check whether grammar is LL(1).
4. Practice epsilon-heavy grammar examples.
5. Explain why FOLLOW is needed for epsilon productions.

14. Final Cheat Sheet

Item	Answer
Core definition	FIRST begins; FOLLOW comes after
Why it matters	Required for predictive parsing
Most asked questions	epsilon, \$, LL(1) table
Common comparison	FIRST vs FOLLOW
One-line answer	FIRST chooses starts; FOLLOW handles what comes next

Abstract Syntax Tree

1. Overview

Definition

An **Abstract Syntax Tree**, or **AST**, is a tree representation of the meaningful structure of source code, excluding unnecessary grammar details.

Why It Matters

The AST is the central structure used by compilers, interpreters, linters, transpilers, IDEs, and static analyzers.

Where It Is Used in Real Systems

System	Use
Compiler	Semantic analysis and code generation
Browser	JavaScript parsing
IDE	Refactoring and autocomplete
Linters	Detecting bad code patterns
Transpiler	TypeScript to JavaScript, Babel

Why Interviewers Ask

AST is a practical bridge between parsing and semantic/code-generation phases.

2. Core Idea

Intuition

An AST keeps what the program means, not every syntax detail.

Real-World Analogy

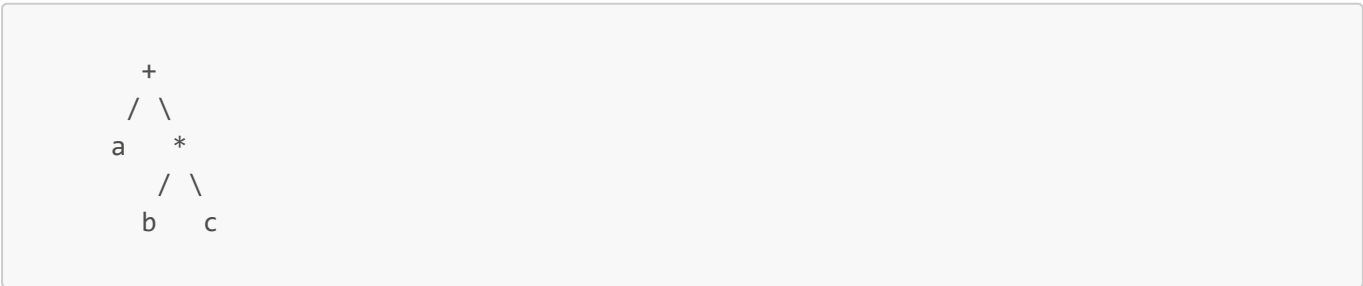
A parse tree is like a full grammatical breakdown of a sentence. An AST is like the important meaning summary.

Small Example

Expression:

```
a + b * c
```

AST:



It shows multiplication happens before addition.

Step-by-Step Explanation

1. Parser reads tokens.
2. Parser checks grammar.
3. Compiler builds parse tree or directly builds AST.
4. AST removes punctuation and redundant grammar nodes.
5. Later phases use AST for semantic checks and IR generation.

3. Important Subtopics

Parse Tree vs AST

Parse tree includes all grammar productions. AST keeps essential structure.

Why it matters: AST is smaller and more useful.

Example: parentheses may not appear as AST nodes.

Interview angle: difference between parse tree and AST.

AST Nodes

Each node represents a construct like expression, statement, function, or declaration.

Why it matters: compiler traverses nodes.

Example: BinaryExpression, IfStatement.

Interview angle: node types.

Tree Traversal

Compiler walks the AST to analyze or transform code.

Why it matters: semantic analysis and optimization.

Example: visit all variables to check declarations.

Interview angle: visitor pattern.

AST Transformation

Changing AST to another AST or IR.

Why it matters: used in transpilers and optimizers.

Example: Babel transforms modern JS syntax to older JS.

Interview angle: source-to-source compilation.

4. Real-World Example

Babel parses JavaScript into an AST, applies transformations, and generates JavaScript again. For example, it may convert modern syntax into older syntax for browser compatibility.

5. Diagrams / Mental Models

Parse tree may include grammar details:

```
Expr  
-> Expr + Term  
-> Term * Factor
```

AST focuses on meaning:

```
BinaryExpr(+)  
  left: Identifier(a)  
  right: BinaryExpr(*)  
    left: Identifier(b)  
    right: Identifier(c)
```

6. Common Interview Questions

1. What is an AST?

Answer: A tree representation of the abstract syntactic structure of source code.

Expected points: meaningful nodes, used after parsing.

Common mistake: saying it contains every token.

2. How is AST different from parse tree?

Answer: Parse tree includes all grammar details; AST removes unnecessary syntax and keeps semantic structure.

Expected points: compactness.

Common mistake: treating them as identical.

3. Why is AST useful?

Answer: It is easier to analyze, transform, and generate code from than raw tokens or parse trees.

Expected points: semantic analysis, IR generation.

Common mistake: saying it is only for visualization.

4. Does AST include parentheses?

Answer: Usually not as separate nodes; their effect appears in tree structure.

Expected points: abstraction.

Common mistake: saying all punctuation stays.

5. What are AST nodes?

Answer: Objects or records representing constructs like expressions, statements, declarations, and functions.

Expected points: node type and children.

Common mistake: thinking every character is a node.

6. Who creates the AST?

Answer: Usually the parser, sometimes with help from syntax-directed translation.

Expected points: parsing phase.

Common mistake: saying lexer creates AST.

7. How does compiler use AST for type checking?

Answer: It traverses expression and declaration nodes, checking types and scopes using symbol table information.

Expected points: traversal.

Common mistake: saying type checking happens during lexing.

8. What is AST traversal?

Answer: Visiting nodes recursively to analyze or transform them.

Expected points: DFS/visitor.

Common mistake: unrelated graph traversal.

9. What is AST used for in IDEs?

Answer: Autocomplete, refactoring, go-to-definition, diagnostics, and formatting.

Expected points: practical tooling.

Common mistake: saying only compilers use ASTs.

10. Can AST be converted back to source code?

Answer: Yes, transpilers and formatters generate source from AST.

Expected points: code generation.

Common mistake: saying impossible because syntax was lost.

7. Deep-Dive Questions

1. Why is AST called abstract?

Because it abstracts away grammar-specific details and keeps core program structure.

2. How do ASTs represent operator precedence?

Through tree shape. Higher-precedence operations appear deeper in the tree.

3. What is a typed AST?

An AST annotated with type information after semantic analysis.

4. What is AST lowering?

Transforming high-level AST constructs into simpler lower-level forms or IR.

5. Why not generate machine code directly from parse tree?

Parse trees are too detailed and grammar-specific; ASTs are cleaner for analysis and generation.

8. Comparison Tables

Aspect	Parse Tree	AST
Detail	Full grammar details	Essential structure
Size	Larger	Smaller
Includes punctuation	Often yes	Usually no
Used for	Grammar proof/parsing	Semantic analysis/codegen
Example	Nonterminals for every rule	Expression/statement nodes

9. Common Mistakes

- Saying AST is same as parse tree.
- Thinking lexer creates AST.
- Thinking AST contains machine code.
- Forgetting AST preserves precedence.
- Assuming AST always stores comments.

10. Edge Cases / Special Cases

- Some tools preserve comments in AST-like structures.
- Error-tolerant ASTs are used by IDEs.
- Parentheses may be stored for formatting tools.
- AST may be typed or untyped.
- Some compilers lower AST into multiple IR stages.

11. How to Explain in Interview

"An AST is a tree that represents the meaningful structure of code. Unlike a parse tree, it removes unnecessary grammar details like punctuation and helper nonterminals. Compilers use it for semantic analysis, transformations, and generating intermediate code."

12. Quick Revision Notes

- AST = abstract syntax structure.
- Smaller than parse tree.
- Used after parsing.
- Nodes represent constructs.
- Tree shape captures precedence.

13. Practice Tasks

1. Draw AST for `a + b * c`.
2. Compare parse tree and AST for an expression.
3. Build a simple AST class for arithmetic expressions.
4. Traverse AST to evaluate expression.
5. Convert AST to three-address code.

14. Final Cheat Sheet

Item	Answer
Core definition	Tree of meaningful source-code structure
Why it matters	Used for analysis, transformation, and code generation
Most asked questions	AST vs parse tree, uses, traversal
Common comparison	Parse tree vs AST
One-line answer	AST keeps what code means, not every grammar detail

Three-Address Code

1. Overview

Definition

Three-address code, or **TAC**, is an intermediate representation where each instruction has at most three addresses: usually two operands and one result.

Example:

```
t1 = b * c
t2 = a + t1
x = t2
```

Why It Matters

TAC makes optimization and code generation easier because complex expressions are broken into simple steps.

Where It Is Used in Real Systems

Compilers use TAC-like IRs for optimization, analysis, and target code generation. LLVM IR is not exactly TAC, but it follows similar low-level intermediate-code ideas.

Why Interviewers Ask

This tests whether you understand intermediate code generation.

2. Core Idea

Intuition

Break a complex expression into simple instructions.

Real-World Analogy

Instead of doing a big calculation mentally, write intermediate results on paper.

Small Example

Expression:

```
x = a + b * c - d;
```

TAC:

```
t1 = b * c
t2 = a + t1
t3 = t2 - d
x = t3
```

Step-by-Step Explanation

1. Respect precedence.
2. Generate temporary variables.
3. Emit one operation per instruction.
4. Use temporaries in later instructions.

3. Important Subtopics

Temporary Variables

Compiler-generated names like `t1`, `t2`.

Why it matters: store intermediate results.

Example: `t1 = b * c`.

Interview angle: expression translation.

Quadruples

Representation with operator, argument1, argument2, result.

Example:

op	arg1	arg2	result
*	b	c	t1

Interview angle: TAC representation.

Triples

Representation using instruction positions instead of named temporary results.

Why it matters: saves explicit temporary names.

Example: `(0): *, b, c`.

Interview angle: quadruples vs triples.

Control Flow TAC

TAC can represent branches and labels.

Example:

```
if x < y goto L1
goto L2
L1: z = 1
L2:
```

Interview angle: translating if/while.

4. Real-World Example

A compiler for C may translate:

```
if (a < b) x = a + b;
```

into:

```

if a < b goto L1
goto L2
L1: t1 = a + b
x = t1
L2:

```

This IR is easier to optimize and convert to assembly.

5. Diagrams / Mental Models

AST:



TAC:

```

t1 = b * c
t2 = a + t1
x = t2

```

6. Common Interview Questions

1. What is three-address code?

Answer: An intermediate representation where each instruction has at most two operands and one result.

Expected points: simple operations, temporaries.

Common mistake: saying exactly three variables always.

2. Why use TAC?

Answer: It simplifies optimization and code generation.

Expected points: machine-independent IR.

Common mistake: saying it is final assembly.

3. Convert $x = a + b * c$ to TAC.

Answer:

```

t1 = b * c
t2 = a + t1

```

```
x = t2
```

Expected points: precedence.

Common mistake: adding before multiplying.

4. What are quadruples?

Answer: A TAC representation with fields op, arg1, arg2, result.

Expected points: table format.

Common mistake: confusing with four-address code.

5. What are triples?

Answer: TAC representation where results are referred to by instruction positions.

Expected points: no explicit result field.

Common mistake: saying triples have three operands.

6. How are conditionals represented?

Answer: Using conditional jumps, unconditional jumps, and labels.

Expected points: control flow.

Common mistake: only handling expressions.

7. What are temporary variables?

Answer: Compiler-generated variables used for intermediate results.

Expected points: not source variables.

Common mistake: thinking programmer declares them.

8. Is TAC machine-independent?

Answer: Mostly yes; it abstracts away specific registers and instructions.

Expected points: IR.

Common mistake: saying tied to x86.

9. How does TAC help optimization?

Answer: Simple instructions make it easier to detect constants, dead code, common subexpressions, and data dependencies.

Expected points: analysis-friendly.

Common mistake: saying it automatically optimizes.

10. Is TAC same as bytecode?

Answer: No. TAC is an intermediate compiler representation; bytecode is usually executable by a VM.

Expected points: distinction.

Common mistake: treating all IR as bytecode.

7. Deep-Dive Questions

1. How does TAC represent loops?

Using labels and jumps.

2. How does TAC handle arrays?

By computing addresses/index offsets and using load/store-like operations.

3. What is indirect triple?

A representation that uses pointers to triples, making code movement easier.

4. Why is TAC good for data-flow analysis?

Each instruction defines and uses a small number of values, making dependencies clear.

5. How is TAC related to SSA?

SSA can be seen as a disciplined IR form where each variable is assigned once, often based on TAC-like operations.

8. Comparison Tables

Aspect	Quadruples	Triples
Fields	op, arg1, arg2, result	op, arg1, arg2
Result naming	Explicit temporary	Instruction position
Code movement	Easier	Harder
Space	More	Less
Example result	<code>t1</code>	<code>(0)</code>

9. Common Mistakes

- Ignoring operator precedence.
- Thinking TAC is assembly.
- Forgetting labels for control flow.
- Thinking every TAC instruction must have exactly three addresses.
- Confusing triples with three-address code itself.

10. Edge Cases / Special Cases

- Unary operations use fewer addresses.
- Assignments use one source and one destination.
- Function calls require parameters and return handling.
- Boolean expressions may use short-circuit jumps.
- Array access needs address calculation.

11. How to Explain in Interview

"Three-address code is an intermediate representation where each instruction performs one simple operation using at most two operands and one result. It breaks complex expressions into temporary assignments, which makes optimization and target code generation easier."

12. Quick Revision Notes

- TAC = simple IR.
- At most three addresses.
- Uses temporaries.
- Useful for optimization.
- Control flow uses labels and jumps.

13. Practice Tasks

1. Convert $x = (a + b) * (c - d)$ to TAC.
2. Write TAC for an if statement.
3. Write TAC for a while loop.
4. Represent TAC using quadruples.
5. Identify dead code in TAC.

14. Final Cheat Sheet

Item	Answer
Core definition	IR with simple instructions using at most three addresses
Why it matters	Simplifies optimization and code generation
Most asked questions	expression to TAC, quadruples/triples
Common comparison	Quadruples vs triples
One-line answer	TAC breaks complex code into simple compiler-friendly steps

Activation Record

1. Overview

Definition

An **activation record**, also called a **stack frame**, is a block of memory created for one function call. It stores information needed to execute the function and return to the caller.

Why It Matters

Function calls, recursion, local variables, parameters, and return addresses all depend on activation records.

Where It Is Used in Real Systems

Operating systems, compilers, debuggers, runtime systems, exception handling, profilers, and stack traces use activation records.

Why Interviewers Ask

This connects compiler design with OS, memory layout, recursion, and function-call mechanics.

2. Core Idea

Intuition

Every function call gets its own workspace on the call stack.

Real-World Analogy

Imagine each function call as a folder on a desk. The folder contains local notes, input values, and the return address. When the function finishes, the folder is removed.

Small Example

```
int add(int a, int b) {  
    int sum = a + b;  
    return sum;  
}
```

Activation record may store:

```
return address  
old frame pointer  
parameters a, b  
local variable sum  
temporary values
```

Step-by-Step Explanation

1. Caller passes arguments.
2. Function call instruction saves return address.
3. Callee creates stack frame.
4. Local variables are allocated.

5. Function executes.
6. Return value is placed in register or stack.
7. Frame is destroyed.
8. Control returns to caller.

3. Important Subtopics

Return Address

Address where execution resumes after function returns.

Why it matters: enables function calls.

Example: after `add(2, 3)`, continue at next instruction.

Interview angle: stack overflow/security.

Parameters

Input values passed to a function.

Why it matters: function communication.

Example: `a`, `b`.

Interview angle: stack vs register passing.

Local Variables

Variables declared inside function.

Why it matters: each call needs separate storage.

Example: `sum`.

Interview angle: recursion.

Dynamic Link

Pointer to caller's frame.

Why it matters: restores previous frame.

Example: saved frame pointer.

Interview angle: stack unwinding.

Temporaries

Intermediate values during expression evaluation.

Why it matters: compiler needs storage when registers are insufficient.

Example: temporary result of `a + b`.

Interview angle: register spilling.

4. Real-World Example

When a program crashes, the debugger prints a stack trace. Each line corresponds to an activation record showing which function called which function.

Example:

```
main()
-> process()
   -> parse()
      -> error()
```

5. Diagrams / Mental Models

```
Call Stack

High Address
+-----+
| main frame      |
+-----+
| process frame   |
+-----+
| parse frame     | <- current function
+-----+
Low Address
```

Typical activation record:

```
+-----+
| Parameters      |
| Return address  |
| Old frame ptr   |
| Local variables |
| Temporaries     |
+-----+
```

6. Common Interview Questions

1. What is an activation record?

Answer: Memory block for a function call containing return address, parameters, locals, saved registers, and temporaries.

Expected points: stack frame.

Common mistake: saying it stores only local variables.

2. Why is activation record needed?

Answer: To manage function execution state and return correctly.

Expected points: calls, recursion.

Common mistake: ignoring return address.

3. Where is activation record stored?

Answer: Usually on the call stack, though optimizations may use registers or heap in special cases.

Expected points: stack frame.

Common mistake: always heap.

4. What happens during function call?

Answer: Arguments are passed, return address saved, frame created, locals allocated, function executes, frame removed.

Expected points: call sequence.

Common mistake: vague "function starts."

5. How does recursion work with activation records?

Answer: Each recursive call gets a separate activation record with its own locals and return address.

Expected points: separate frames.

Common mistake: thinking same local variable is reused.

6. What is stack overflow?

Answer: When too many activation records are pushed and stack memory is exhausted.

Expected points: recursion/common cause.

Common mistake: confusing with heap overflow.

7. What is frame pointer?

Answer: A register/pointer used to access fixed locations in the current stack frame.

Expected points: stack layout.

Common mistake: saying it stores return value.

8. What are saved registers?

Answer: Registers preserved across function calls according to calling convention.

Expected points: caller/callee saved.

Common mistake: ignoring calling convention.

9. What is stack unwinding?

Answer: Removing stack frames during exception handling or function returns.

Expected points: exceptions/debugging.

Common mistake: only normal returns.

10. What is calling convention?

Answer: Rules for passing arguments, returning values, and saving registers.

Expected points: ABI.

Common mistake: language-specific only.

7. Deep-Dive Questions

1. Can activation records be allocated on heap?

Yes, if closures or coroutines require frames to outlive the function call.

2. What is tail-call optimization?

An optimization where the current frame is reused for a final function call.

3. How do closures affect activation records?

Captured variables may need heap allocation if they outlive the stack frame.

4. How do exceptions use stack frames?

Runtime unwinds frames until it finds a matching exception handler.

5. What is register spilling?

When there are not enough registers, values are stored in the activation record.

8. Comparison Tables

Aspect	Stack	Heap
Used for	Activation records, locals	Dynamic allocation
Lifetime	Function call scoped	Programmer/runtime controlled
Speed	Usually faster	Usually slower
Allocation	Push/pop	Allocator/GC

Aspect	Stack	Heap
Error	Stack overflow	Memory leak/out of memory

9. Common Mistakes

- Thinking activation record stores only variables.
- Forgetting return address.
- Thinking recursion reuses one frame.
- Confusing stack and heap.
- Ignoring calling conventions.

10. Edge Cases / Special Cases

- Tail-call optimization may avoid new frame.
- Inline functions may not create normal frames.
- Closures may move captured data to heap.
- Optimized builds may omit frame pointer.
- Coroutines may store frames differently.

11. How to Explain in Interview

"An activation record is the stack frame for one function call. It stores parameters, return address, saved registers, local variables, and temporaries. Each call gets its own frame, which is why recursion works and why deep recursion can cause stack overflow."

12. Quick Revision Notes

- Activation record = stack frame.
- Stores return address, params, locals, saved registers.
- One frame per call.
- Recursion uses multiple frames.
- Calling convention defines layout rules.

13. Practice Tasks

1. Draw stack frames for recursive factorial.
2. Trace function call and return.
3. Identify local variables and parameters in a C function.
4. Explain stack overflow using recursion.
5. Compare stack and heap allocation.

14. Final Cheat Sheet

Item	Answer
Core definition	Memory block for one function call
Why it matters	Supports calls, recursion, returns, locals

Item	Answer
Most asked questions	stack frame contents, recursion, stack overflow
Common comparison	Stack vs heap
One-line answer	An activation record is a function call's workspace on the stack

Register Allocation

1. Overview

Definition

Register allocation is the compiler process of deciding which program values should be stored in CPU registers and which should be stored in memory.

Why It Matters

Registers are much faster than memory but limited in number. Good allocation improves performance.

Where It Is Used in Real Systems

Native compilers, JIT compilers, embedded systems, game engines, high-performance servers, and mobile runtimes all rely on register allocation.

Why Interviewers Ask

It tests whether you understand compiler backend optimization and hardware constraints.

2. Core Idea

Intuition

Keep frequently used values in fast storage.

Real-World Analogy

Registers are like items on your desk. Memory is like a cupboard. You keep current work on the desk because reaching the cupboard repeatedly is slow.

Small Example

```
t1 = a + b
t2 = t1 * c
```

If registers are available:

```
R1 = a
R2 = b
R1 = R1 + R2
R3 = c
R1 = R1 * R3
```

If registers are limited, some values are spilled to memory.

Step-by-Step Explanation

1. Find variable live ranges.
2. Build interference information.
3. Assign registers to values that do not overlap.
4. Spill values to memory if registers are insufficient.
5. Generate final instructions.

3. Important Subtopics

Live Range

The span during which a value may be used in future.

Why it matters: overlapping live ranges cannot share a register.

Example: if **a** and **b** are both needed at same time, they need different registers.

Interview angle: liveness analysis.

Interference Graph

Graph where nodes are variables and edges mean they are live at the same time.

Why it matters: register allocation resembles graph coloring.

Example: edge between **x** and **y** means no same register.

Interview angle: graph coloring.

Spilling

Moving values from registers to memory.

Why it matters: needed when registers are insufficient.

Example: store temporary on stack.

Interview angle: performance cost.

Caller-Saved and Callee-Saved Registers

Calling convention decides who preserves registers.

Why it matters: function calls affect allocation.

Example: callee must restore callee-saved register.

Interview angle: ABI awareness.

4. Real-World Example

In a tight loop:

```
for (int i = 0; i < n; i++) sum += arr[i];
```

A good compiler keeps `i`, `n`, `sum`, and base address of `arr` in registers to avoid repeated memory loads.

5. Diagrams / Mental Models

Variables live at same time:

```
a ----|
b -----|
c  ---|
```

a interferes with b
b interferes with c

Interference graph:

```
a ---- b
      /
      c
```

6. Common Interview Questions

1. What is register allocation?

Answer: Assigning program values to CPU registers or memory.

Expected points: limited registers, performance.

Common mistake: saying it creates registers.

2. Why is register allocation important?

Answer: Register access is faster than memory access.

Expected points: performance.

Common mistake: ignoring limited register count.

3. What is spilling?

Answer: Storing a value in memory when no register is available.

Expected points: stack slots, performance cost.

Common mistake: treating spill as optimization benefit.

4. What is live range?

Answer: The program region where a value is needed for future use.

Expected points: liveness.

Common mistake: confusing with variable scope.

5. What is interference graph?

Answer: A graph where variables connected by edges cannot share a register.

Expected points: graph coloring.

Common mistake: thinking it is control-flow graph.

6. How is graph coloring related?

Answer: Registers are colors; variables connected by edges need different colors.

Expected points: limited colors/registers.

Common mistake: saying graph coloring always finds easy solution.

7. What happens if not enough registers?

Answer: Compiler spills some values to memory.

Expected points: spill choice.

Common mistake: program cannot run.

8. What is register pressure?

Answer: The demand for registers at a program point.

Expected points: many live values.

Common mistake: hardware voltage/CPU pressure.

9. Difference between allocation and assignment?

Answer: Allocation decides which values get registers; assignment chooses exact physical registers.

Expected points: phases may be combined.

Common mistake: no distinction.

10. Why do function calls complicate allocation?

Answer: Calls may overwrite caller-saved registers and require preserving callee-saved registers.

Expected points: calling convention.

Common mistake: ignoring calls.

7. Deep-Dive Questions

1. Why is optimal register allocation hard?

It is related to graph coloring, which is computationally hard in general.

2. What is linear scan allocation?

A simpler register allocation algorithm that scans live intervals, common in JIT compilers.

3. Why do JITs often use linear scan?

It is fast and good enough for runtime compilation.

4. How does SSA help register allocation?

SSA makes def-use relationships clearer and can simplify liveness reasoning.

5. What is coalescing?

Removing unnecessary move instructions by assigning source and destination to the same register when safe.

8. Comparison Tables

Aspect	Register	Memory
Speed	Very fast	Slower
Quantity	Very limited	Large
Access	CPU direct	Load/store needed
Managed by	Compiler/backend	Runtime/hardware
Example	RAX, RBX	Stack slot

Aspect	Graph Coloring	Linear Scan
Quality	Usually better	Usually decent
Speed	Slower	Faster
Used in	AOT compilers	JIT compilers
Complexity	Higher	Lower

9. Common Mistakes

- Confusing live range with lexical scope.
- Thinking variables always stay in memory.
- Thinking unlimited registers exist.
- Ignoring spilling cost.
- Confusing interference graph with control-flow graph.

10. Edge Cases / Special Cases

- Some architectures have special-purpose registers.
- Function calls may force saves/restores.
- SIMD/vector registers are separate resources.
- Spilling inside loops is especially expensive.
- Inline assembly can constrain registers.

11. How to Explain in Interview

"Register allocation is the compiler backend task of mapping variables and temporaries to limited CPU registers. Since registers are faster than memory, good allocation improves performance. If there are not enough registers, values are spilled to memory."

12. Quick Revision Notes

- Registers are fast but limited.
- Live ranges determine sharing.
- Interference graph models conflicts.
- Spilling stores values in memory.
- Register pressure means too many live values.

13. Practice Tasks

1. Find live ranges in a TAC sequence.
2. Draw an interference graph.
3. Color a small graph with two registers.
4. Identify where spilling is needed.
5. Compare linear scan and graph coloring.

14. Final Cheat Sheet

Item	Answer
Core definition	Mapping program values to registers
Why it matters	Reduces memory access and improves speed
Most asked questions	spilling, liveness, interference graph
Common comparison	Register vs memory
One-line answer	Register allocation decides what gets fast CPU storage

Dead-Code Elimination

1. Overview

Definition

Dead-code elimination, or **DCE**, is a compiler optimization that removes code whose result is never used or code that can never execute.

Why It Matters

Removing dead code improves performance, reduces binary size, and simplifies later optimization.

Where It Is Used in Real Systems

Compilers, JavaScript bundlers, minifiers, linkers, database query optimizers, and static analyzers use dead-code elimination.

Why Interviewers Ask

It tests whether you understand program analysis and semantics-preserving optimization.

2. Core Idea

Intuition

If code does not affect observable program behavior, remove it.

Real-World Analogy

If a recipe says "chop onions" but onions are never used in the dish, that step can be removed.

Small Example

```
int x = 5;  
x = 10;  
printf("%d", x);
```

`x = 5` is dead because its value is overwritten before being used.

Optimized:

```
int x = 10;  
printf("%d", x);
```

Step-by-Step Explanation

1. Analyze which values are used later.
2. Mark instructions that affect output, memory, I/O, or control flow.
3. Remove assignments whose results are unused.
4. Repeat because removing code can make more code dead.

3. Important Subtopics

Unreachable Code

Code that can never execute.

Why it matters: safe to remove.

Example:

```
return;  
x = 5;
```

Interview angle: control-flow analysis.

Dead Store

Assignment to a variable that is overwritten before being read.

Why it matters: common optimization.

Example: `x = 1; x = 2; print(x);`.

Interview angle: liveness analysis.

Side Effects

Observable effects like I/O, memory writes, exceptions, volatile access.

Why it matters: code with side effects may not be removable.

Example: `printf("hi")` cannot be removed just because return value unused.

Interview angle: semantic preservation.

Liveness Analysis

Determines whether a value may be used in future.

Why it matters: foundation for DCE.

Example: if `t1` is never used, assignment to `t1` is dead.

Interview angle: data-flow analysis.

4. Real-World Example

JavaScript bundlers remove unused exports:

```
export function used() {}  
export function unused() {}
```

If only `used()` is imported, bundlers can remove `unused()` during tree shaking, reducing frontend bundle size.

5. Diagrams / Mental Models

```
t1 = a + b      dead if t1 never used  
t2 = c + d      live if used by print  
print(t2)       side effect, keep
```

Control-flow example:

```
return x  
  |  
  v  
y = 10   unreachable
```

6. Common Interview Questions

1. What is dead-code elimination?

Answer: Removing code that does not affect observable program behavior.

Expected points: unused or unreachable code.

Common mistake: removing code just because it looks unnecessary.

2. What is dead code?

Answer: Code whose result is unused or code that cannot execute.

Expected points: dead stores and unreachable code.

Common mistake: only unreachable code.

3. Give example of dead store.

Answer: `x = 1; x = 2; print(x);` makes `x = 1` dead.

Expected points: overwritten before read.

Common mistake: saying `x = 2` dead.

4. Can function calls be removed if return value unused?

Answer: Only if compiler proves they have no side effects.

Expected points: side effects.

Common mistake: always yes.

5. Why are side effects important?

Answer: Removing side-affecting code changes program behavior.

Expected points: I/O, memory writes, exceptions.

Common mistake: considering only returned values.

6. What analysis helps DCE?

Answer: Liveness analysis and control-flow analysis.

Expected points: data-flow.

Common mistake: lexical analysis.

7. Is unreachable code always removable?

Answer: Usually yes, if it truly cannot execute and has no required compile-time effect.

Expected points: language rules.

Common mistake: ignoring special language semantics.

8. What is tree shaking?

Answer: A module-level form of dead-code elimination that removes unused exports/imports.

Expected points: bundlers.

Common mistake: saying it only removes comments.

9. Does DCE change output?

Answer: It should not change observable behavior.

Expected points: semantics-preserving.

Common mistake: saying optimization may change logic.

10. Why repeat DCE?

Answer: Removing one instruction may make another instruction dead.

Expected points: fixed-point optimization.

Common mistake: only one pass always enough.

7. Deep-Dive Questions

1. What is observable behavior?

Output, memory writes visible outside, volatile access, exceptions, I/O, and externally visible state changes.

2. Why are volatile variables special?

Reads/writes to volatile variables are observable and usually cannot be removed.

3. How does SSA help DCE?

SSA makes each value definition explicit, so unused definitions are easier to identify.

4. What is aggressive DCE?

It starts from operations known to be live and removes everything not proven live.

5. Can infinite loops be removed?

Carefully. Infinite loops may be observable depending on language rules, volatile operations, or termination semantics.

8. Comparison Tables

Aspect	Dead Code	Unreachable Code
Meaning	Result not used or no effect	Cannot execute
Example	<code>x=1; x=2;</code>	code after return
Analysis	Liveness/data-flow	Control-flow
Removal	If no side effects	If truly unreachable

9. Common Mistakes

- Ignoring side effects.
- Removing function calls unsafely.
- Thinking comments removal is DCE.
- Confusing dead code with slow code.
- Forgetting volatile/exception behavior.

10. Edge Cases / Special Cases

- Function calls may throw exceptions.
- Memory writes through pointers may be observable.
- Volatile reads/writes must be preserved.
- Debug builds may keep dead-looking code.
- Reflection/dynamic loading can complicate module-level DCE.

11. How to Explain in Interview

"Dead-code elimination removes code that cannot affect observable behavior, such as unreachable statements or assignments whose values are never used. The compiler must be careful with side effects like I/O, memory writes, volatile access, and exceptions."

12. Quick Revision Notes

- DCE removes unused/unreachable code.
- Must preserve observable behavior.
- Dead store: overwritten before read.
- Side effects prevent removal.
- Liveness analysis helps DCE.

13. Practice Tasks

1. Identify dead assignments in TAC.
2. Remove unreachable code after return.
3. Mark side-effecting instructions.
4. Perform one DCE pass and repeat.
5. Explain tree shaking in JavaScript.

14. Final Cheat Sheet

Item	Answer
Core definition	Removes code with no observable effect
Why it matters	Improves performance and size
Most asked questions	dead store, side effects, liveness
Common comparison	Dead code vs unreachable code
One-line answer	DCE deletes code only when behavior stays the same

SSA Form

1. Overview

Definition

Static Single Assignment, or **SSA**, is an intermediate representation where each variable is assigned exactly once.

When control-flow paths merge, SSA uses **phi functions** to choose the correct value.

Why It Matters

SSA makes data dependencies explicit, simplifying optimization such as constant propagation, dead-code elimination, and register allocation.

Where It Is Used in Real Systems

Modern compilers and JITs use SSA or SSA-like IRs, including LLVM, GCC internals, JVM JITs, and JavaScript engines.

Why Interviewers Ask

SSA is an advanced compiler optimization concept. Interviewers ask it to test depth beyond basic compiler phases.

2. Core Idea

Intuition

Instead of reassigning the same variable, give each new value a fresh name.

Real-World Analogy

Versioned documents:

```
x1 = first version  
x2 = updated version  
x3 = final version
```

You never overwrite; you create a new version.

Small Example

Original:

```
x = 1;  
x = x + 2;  
y = x * 3;
```

SSA:

```
x1 = 1  
x2 = x1 + 2  
y1 = x2 * 3
```

With branch:

```
if (cond) x = 1;  
else x = 2;  
y = x + 3;
```

SSA:

```
if cond goto L1 else L2  
L1: x1 = 1; goto L3  
L2: x2 = 2; goto L3  
L3: x3 = phi(x1, x2)  
y1 = x3 + 3
```

Step-by-Step Explanation

1. Rename each assignment to a new variable version.
2. Track uses to refer to the correct version.
3. Insert phi functions at control-flow merge points.
4. Run optimizations more easily.
5. Later lower SSA back to machine-level code.

3. Important Subtopics

Single Assignment

Each variable version is assigned once.

Why it matters: def-use chains are clear.

Example: `x1`, `x2`, `x3`.

Interview angle: not same as immutable source variables.

Phi Function

A pseudo-operation at control-flow merge points.

Why it matters: selects value depending on predecessor block.

Example: `x3 = phi(x1, x2)`.

Interview angle: why phi is needed.

Control-Flow Graph

Graph of basic blocks and branches.

Why it matters: phi placement depends on merge points.

Example: if-else join block.

Interview angle: dominance frontier.

Def-Use Chain

Links each definition to its uses.

Why it matters: simplifies optimizations.

Example: all uses of `x2` point to one definition.

Interview angle: constant propagation and DCE.

4. Real-World Example

LLVM IR uses SSA-style values:

```
%1 = add i32 %a, %b
%2 = mul i32 %1, 2
```

Each `%` value is assigned once. This helps LLVM apply optimizations across many programming languages.

5. Diagrams / Mental Models

```

      cond
     /   \
x1 = 1  x2 = 2
     \   /
      phi
       |
x3 = phi(x1, x2)
       |
y1 = x3 + 3

```

Variable versioning:

```

x = 1      -> x1 = 1
x = x + 2  -> x2 = x1 + 2
x = x * 4  -> x3 = x2 * 4

```

6. Common Interview Questions

1. What is SSA form?

Answer: An IR form where each variable version is assigned exactly once.

Expected points: variable renaming, phi functions.

Common mistake: saying source variables cannot change.

2. Why is SSA useful?

Answer: It makes data flow explicit and simplifies optimization.

Expected points: DCE, constant propagation.

Common mistake: saying it only saves memory.

3. What is a phi function?

Answer: A pseudo-operation that selects a value based on which control-flow path reached a block.

Expected points: merge points.

Common mistake: treating phi as normal runtime function call.

4. Convert simple reassignment to SSA.

Answer: `x=1; x=x+1;` becomes `x1=1; x2=x1+1;`.

Expected points: fresh version per assignment.

Common mistake: reusing `x1`.

5. Where are phi nodes inserted?

Answer: At control-flow merge points where different definitions may reach the same use.

Expected points: join blocks.

Common mistake: placing phi after every assignment.

6. Does SSA exist in source code?

Answer: Usually no. It is an internal compiler representation.

Expected points: IR.

Common mistake: confusing with functional programming.

7. How does SSA help DCE?

Answer: If a definition has no uses and no side effects, it can be removed easily.

Expected points: def-use clarity.

Common mistake: ignoring side effects.

8. How does SSA help constant propagation?

Answer: Each use has exactly one reaching definition, making constant values easier to track.

Expected points: single definition.

Common mistake: saying constants automatically replace all variables.

9. Is phi function executed?

Answer: It is a compiler IR concept; later it is lowered to moves or register assignments.

Expected points: pseudo-operation.

Common mistake: thinking CPU has phi instruction.

10. Is SSA permanent?

Answer: No. It is usually converted out of SSA before final machine code.

Expected points: lowering.

Common mistake: saying final assembly is SSA.

7. Deep-Dive Questions

1. What is dominance?

A block dominates another if every path to the second block goes through the first.

2. What is dominance frontier?

It identifies where definitions from different paths meet, helping decide phi placement.

3. How is SSA destroyed?

Phi nodes are replaced with move instructions in predecessor blocks or resolved during register allocation.

4. What is pruned SSA?

SSA form that inserts phi nodes only for variables that are actually live.

5. How do loops use phi nodes?

Phi nodes merge initial values and loop-carried updated values.

Example:

```
i1 = 0
loop:
i2 = phi(i1, i3)
i3 = i2 + 1
```

8. Comparison Tables

Aspect	Normal IR	SSA IR
Assignment	Variable can be assigned many times	Each version assigned once
Data flow	Harder to track	Explicit
Optimization	More complex	Easier
Merge handling	Implicit	Phi functions
Final machine code	Closer to machine model	Must be lowered

Aspect	Phi Function	Normal Function
Exists in	Compiler IR	Program/runtime
Purpose	Merge values from control paths	Execute logic
Runtime call	No	Yes
Lowered to	Moves/register choices	Call instructions

9. Common Mistakes

- Thinking SSA means variables in source code cannot be reassigned.
- Forgetting phi functions.
- Placing phi functions everywhere.
- Thinking phi is a CPU instruction.
- Thinking SSA is final machine code.

10. Edge Cases / Special Cases

- Loops require phi nodes for loop-carried variables.
- Memory SSA handles memory dependencies.
- Phi placement can be optimized.
- SSA must handle unreachable blocks carefully.
- Exceptions complicate control flow and phi placement.

11. How to Explain in Interview

"SSA is an intermediate representation where each variable version is assigned exactly once. Reassignments become new versions like `x1`, `x2`, and merge points use phi functions. This makes data flow explicit and helps optimizations like constant propagation and dead-code elimination."

12. Quick Revision Notes

- SSA = Static Single Assignment.
- Each variable version assigned once.
- Phi merges branch values.
- Helps optimizations.
- Lowered before final code.

13. Practice Tasks

1. Convert straight-line code to SSA.
2. Convert if-else code to SSA with phi.
3. Convert a loop to SSA.
4. Identify unused SSA definitions.
5. Explain how phi lowers to moves.

14. Final Cheat Sheet

Item	Answer
Core definition	IR where each variable version is assigned once
Why it matters	Makes data flow and optimization easier
Most asked questions	phi function, branch merge, DCE
Common comparison	Normal IR vs SSA
One-line answer	SSA versions every assignment and uses phi nodes at merges

Additional Placement Revision Map

The topics above are strongly connected. Use this map before interviews:

```
Source Code
  |
  v
Lexical Analysis
  |-- token, lexeme, regex
  v
Parsing
  |-- LL/LR, left recursion, left factoring, FIRST/FOLLOW
  v
AST
  |
  v
Semantic Analysis
  |
  v
Three-Address Code / SSA
  |
  v
Optimization
  |-- dead-code elimination
  v
Code Generation
  |-- register allocation
  v
Runtime
  |-- activation records
```

One-Minute Compiler Design Interview Summary

A compiler translates source code into target code through phases. Lexical analysis converts characters into tokens using regular expressions and automata. Parsing checks token structure using grammar and may use LL or LR techniques. FIRST and FOLLOW help build predictive parsers, while left recursion and left factoring are grammar transformations for top-down parsing. The parser builds an AST, which is used for semantic analysis and intermediate code generation. Three-address code and SSA are intermediate representations that simplify optimization. Optimizations like dead-code elimination remove useless code. The backend performs register allocation and code generation. At runtime, function calls use activation records on the stack.